Zarządzanie Projektem Informatycznym

Sprawozdanie – Laboratorium nr 1

Doświadczenie w projekcie – LocationFinder

Imię i nazwisko: wiktor dąbrowski
Kierunek / grupa: isi 1
Data: 13.10.2026

1. Opis projektu

Uczestniczyłem w zespołowym projekcie LocationFinder, czyli lokalnym, w pełni offline systemie do weryfikacji zdjęć. System analizuje zdjęcia budynków i scen plenerowych pod kątem autentyczności (zdjęcie prawdziwe czy wygenerowane przez AI), metadanych, geolokalizacji, obiektów oraz tablic rejestracyjnych.

Technologie: Python, Django REST Framework, React, PostgreSQL, Celery, Redis i Docker Compose. Do analizy wykorzystaliśmy modele uczenia maszynowego (m.in. klasyfikatory AI vs. real, YOLOv8, EasyOCR) oraz model Gemma uruchamiany lokalnie przez Ollama.

Pracowaliśmy w zespole, w którym kierunek architektury wyznaczał szef zespołu, a poszczególne obszary (np. integrację Django z Celery) prowadziły różne osoby.

2. Przebieg i napotkane problemy

Ograniczenia sprzętowe. Każdy z nas pracował na prywatnym laptopie z ok. 16 GB RAM, a pełny zestaw modeli zbliżał się do tego limitu. Rozwiązaniem było leniwe ładowanie modeli (jednorazowo na proces workera) oraz profile Docker Compose, dzięki którym domyślnie startują tylko baza i aplikacja webowa, a moduły analityczne dopiero na żądanie.

Odrzucone rozwiązania. Integrację z ExifTool zaczęliśmy robić, ale się nie sprawdziła i z niej zrezygnowaliśmy. Podobnie z pełnym snapshotem geokodera Photon (ok. 57 GB), który okazał się za duży. Z projektu usunęliśmy też zbyt złożone metody wykrywania obrazów AI (CNNDetection, UnivFD) na rzecz prostszego zestawu dwóch modeli.

Problem z panelem obserwacji. Pod koniec projektu okazało się, że panel obserwacji opierał się głównie na danych zebranych we wcześniejszej analizie zdjęcia. Gdy użytkownik zadał pytanie o szczegół, który nie znalazł się w opisie wygenerowanym przez system, panel odpowiadał, że informacji nie ma w raporcie, mimo że zdjęcie można było przeanalizować dokładniej. Problem wyszedł dopiero przy testowaniu gotowego systemu, a nie podczas projektowania. Naprawa wymagała zmiany sposobu, w jaki panel korzysta z danych, i zajęła nam sporo czasu, a był to już końcowy etap projektu.

Drobne błędy techniczne. Przykładem jest błąd budowania obrazu Dockera spowodowany złą składnią instrukcji ARG.

3. Wnioski

1. **Scenariusze użycia trzeba testować wcześnie.** Panel sprawdzaliśmy zbyt późno, dopiero na końcu. Gdybyśmy przetestowali go na pytaniach, których nie przewidzieliśmy w raporcie, wada wyszłaby wcześniej i kosztowałaby mniej.
2. **Na etapie projektowania trzeba zapytać, co system zrobi, gdy użytkownik zada pytanie spoza zebranych danych.** Założyliśmy, że raport z analizy wystarczy, a nie uwzględniliśmy realnego sposobu korzystania z panelu.
3. **Elementy łączące kilka modułów wymagają testów integracyjnych od początku.** Łańcuch analiza → raport → panel testowaliśmy dopiero na końcu, a właśnie na styku modułów pojawiają się najdroższe błędy.
4. **Założenia techniczne należy weryfikować na początku.** ExifTool i Photon odrzuciliśmy dopiero po poświęceniu im czasu, a krótki prototyp by to oszczędził.
5. **Ograniczenia zasobów są wymaganiem projektowym.** Limit pamięci RAM trzeba uwzględnić już przy projektowaniu architektury, a nie traktować jako szczegół implementacji.
6. **Prostota jest lepsza niż teoretyczna dokładność.** Usunięcie zbyt skomplikowanych metod uczyniło system łatwiejszym do utrzymania i uruchomienia w całym zespole. Zmiany lepiej wprowadzać małymi krokami, dodając tylko to, czego wymaga bieżący etap.
7. **W zespole potrzebny jest jasny podział odpowiedzialności i wspólne środowisko.** Przypisanie odpowiedzialności za moduły (np. integrację Celery) oraz powtarzalna konfiguracja w Docker Compose ograniczają chaos przy pracy wielu osób.